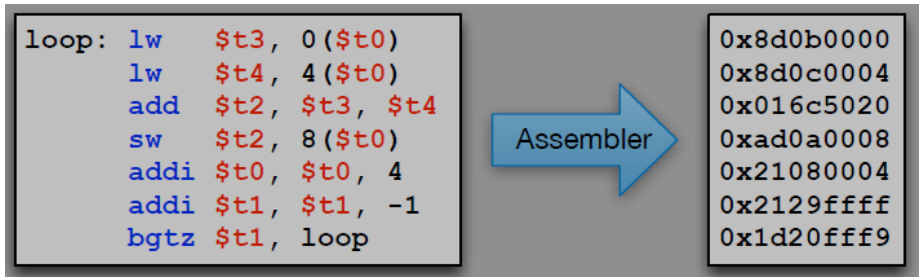


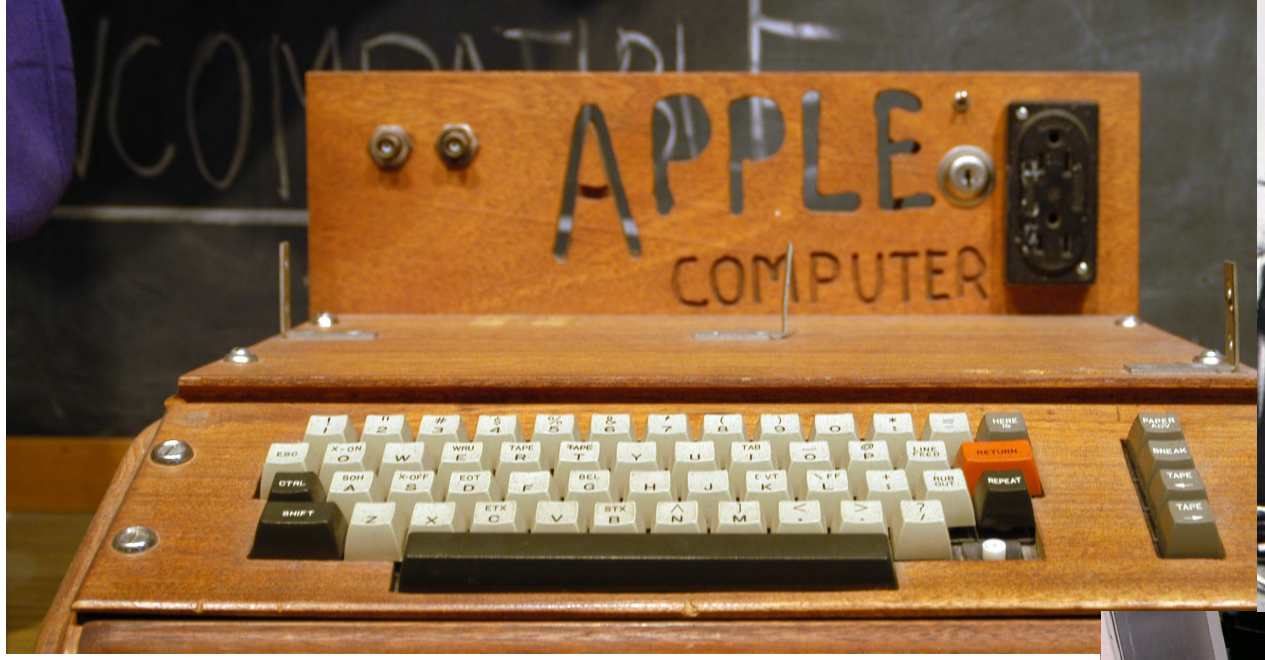
Lecture 07:
CPU Memory Access

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

This Week on "Didja Know Dat?!"



Steve Wozniak and **Steve Job**'s first commercial venture was the **Apple 1** in **1976** using an **8-bit** **MOS 6502 CPU**.

It was built for $500 and initially **sold for $666.66** because Wozniak "*liked repeating digits*" (about $3,800 in today's dollars!)
Keyboard and TV (display) not included! They sold about 200 of them in 10 months (mostly to computer "hobbyists"), thus assuring the continuation of their new company.

Previously, the only other popular "personal" computer was the Altair 8800, which you had to operate with switches (no keyboard!)

Administrative

Current homework due TODAY!

New homework will be out on Canvas soon!

New readings for this week!

- **Handout 3 on Memory Access/Addressing**
- **Handout 4 on Instruction Assembly**
- **Video on Instruction Assembly**

A worksheet file is available for today's lecture on Canvas!

REMINDER: Come to our office hours!! ☺

Quiz #2 is on Wednesday!

- **We will start it at 11 AM SHARP!!!**
- Material is from Lectures **4 (+4B video)**, **5**, **6**, and **7**, and the **Reading:** CS64_Handout 2

Worksheet

iClicker!

What will this program do when you run it on spim?

A. It will print the number **1**

B. It will print the number **11**

C. It will print the number **111**

D. It will print the number **1111**

E. It will not run because of an error in the code

```
.text
main:
    addi $t0, $zero, 1
    addi $t1, $zero, 100
    add $a0, $zero, $t0
joey:
    bgt $t0, $t1, ross
    addi $t2, $t0, 10
    mult $t0, $t2
    mflo $t0
    li $v0, 1
    syscall
    j joey
ross:
    li $v0, 10
    syscall
```

Worksheet

iClicker!!

Which of the following C++ code is the closest equivalent to this MIPS assembly code?

A

```
int a0=3, a1=3, v0=1;
if (a0 < a1) {
    cout << 3;
} else {
    while (a1 > 0) {
        cout << 2*a0;
        a--;
    }
}
```

B

```
int a0=3, a1=3;
if (a0 >= a1) {
    while (a1 > 0) {
        a0 = 2*a0;
        cout << a0;
        a1--;
    }
}
```

C

```
int a0=3, a1=3;
if (a0 >= a1) {
    for (int n = a1; n >= 0; n--) {
        a0 = 2*a0;
        cout << a0;
    }
}
```

```
.text
main:
    addi $v0, $zero, 1
    addi $a0, $zero, 3
    addi $a1, $zero, 3
    slt $t0, $a0, $a1
    bne $t0, $zero, phoebe
rachel:
    ble $a1, $zero, phoebe
    sll $a0, $a0, 1
    addi $a1, $a1, -1
    li $v0, 1
    syscall
    j rachel
phoebe:
    li $v0, 10
    syscall
```

Worksheet

addu vs. add (the "u" stands for "unsigned")

You add 2 **int** types & you get an answer that's beyond the 32b range. What happens??

- ANS: You get an overflow error (V = 1)

Both **add** and **addi** will *raise an overflow error*, if this happens

- ALU takes note of this sort of thing & could pass it on as a *fatal error* in certain cases

addu (and **addiu**) might still *raise an overflow error*, ***but the ALU/CPU will ignore it!***

Moral of the story?

- Use **add/addi** to deal with signed numbers in 2's complement (e.g. regular **int** types)
- Use **addu/addiu** to deal with unsigned **int** types (e.g., memory addresses, indices, …)

Worksheet

Recall: **li** and **la** are both pseudoinstructions

How is `la` different from `li` ???

`li`

- **L**oad **i**mmediate 32-bit value into a register
- Example usage:

```
li $t0, -555111     # put -555111 as 32b value in reg. $t0
```

`la`

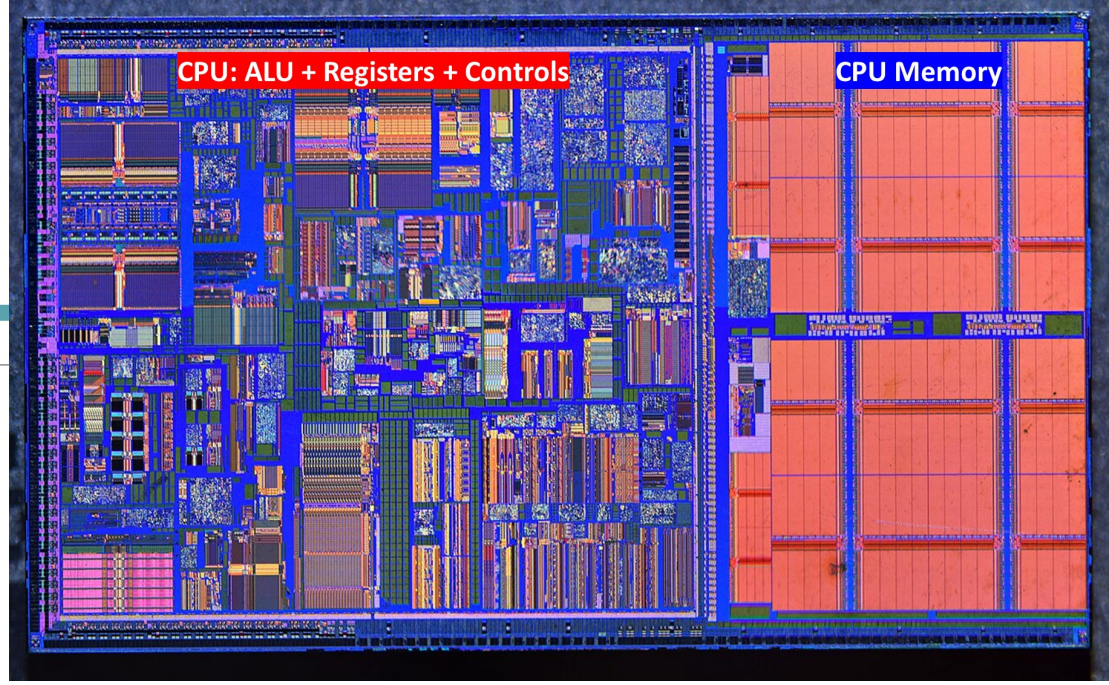
- **L**oad **a**ddress (i.e., memory address label, which *represents* a 32-bit value) into a register
- Example usage:

```
la $t0, ThisAddress
```

- Usually preceded by defining "**`ThisAddress`**" as a MEMORY LABEL ,
  which is a stand-in for an actual 32b memory address
- This memory labeling is done in the **`.data`** part of the program (and **spim** decides what it is)

NOTE: The use AND the syntax of each is *very different* from the other!!!

Larger Data Structures



Recall: CPU **registers** ---vs.--- CPU **memory**

- Where would larger data structures,
  arrays, ***strings***, ***structs/classes***, etc. go?
- Which is *faster* to access: regs or mem? Why?

Some data structures have to be **stored** & **retrieved** in memory

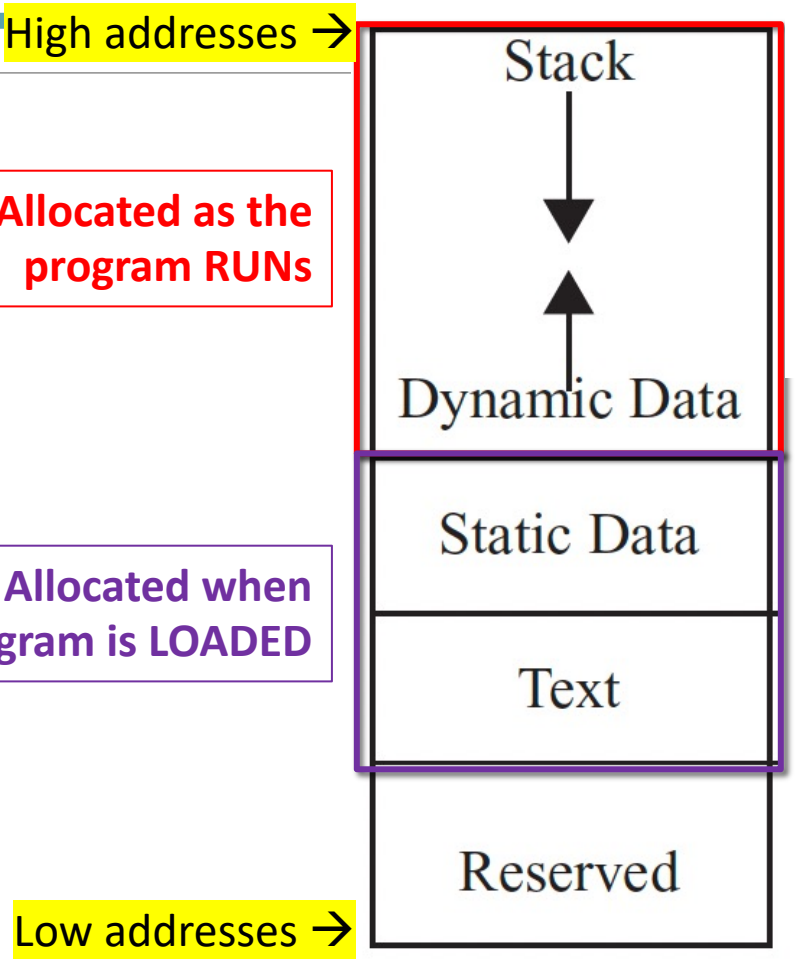
- We need instructions that can
  "*shuttle*" data to/from the CPU and CPU memory (also for RAM, SDD,…)
- We'll see how arrays are done in assembly…

Worksheet

The MemMap

We need to tell the assembler (and the spim simulator) **what bits** should be placed **where** in memory

Behold!!! The Memory Map!



Allocated as the program RUNs

Allocated when program is LOADED

Stack and Heap data that get allocated and deleted throughout the run of the program.
*In assembly, you have to be **very explicit** about using this.*

Initial data to be used in the program (like arrays)

actual program instructions

Global Variables, Arrays, and Strings

Typically, *global variables* are placed directly in memory and **not in** registers

- *Why might this be?*

In **spim**, we use the **.data** directive to set-up CPU Memory for the program

- We can declare "static" variables, their values, and their names (labels) as used in the program
- Storage is then allocated in main CPU memory
- This applies to arrays, structs, etc… INCLUDING strings!!

Worksheet

.data Declaration Types *w/ Examples*

```
var1:   .byte 9          # declare a single byte with value 9
var2:   .half 63         # declare a 16-bit half-word w/ val. 63
var3:   .word 9433       # declare a 32-bit word w/ val. 9433
num1:   .float 3.14      # declare 32-bit floating point number
num2:   .double 6.28     # declare 64-bit floating pointer number
str1:   .ascii "Text"    # declare a string of chars
str3:   .asciiz "Text"   # declare a null-terminated string
str2:   .space 16        # reserve 16 consecutive bytes of space (useful for arrays)
```

Note: **16 bytes** of memory could be **16 characters** or it could be **4 integers** *(for example)*

This is how we reserve values in memory.
We can call them up by loading their memory address into the appropriate registers.

Highlighted ones are the ONLY ones we will use in the CS64 course.

Example

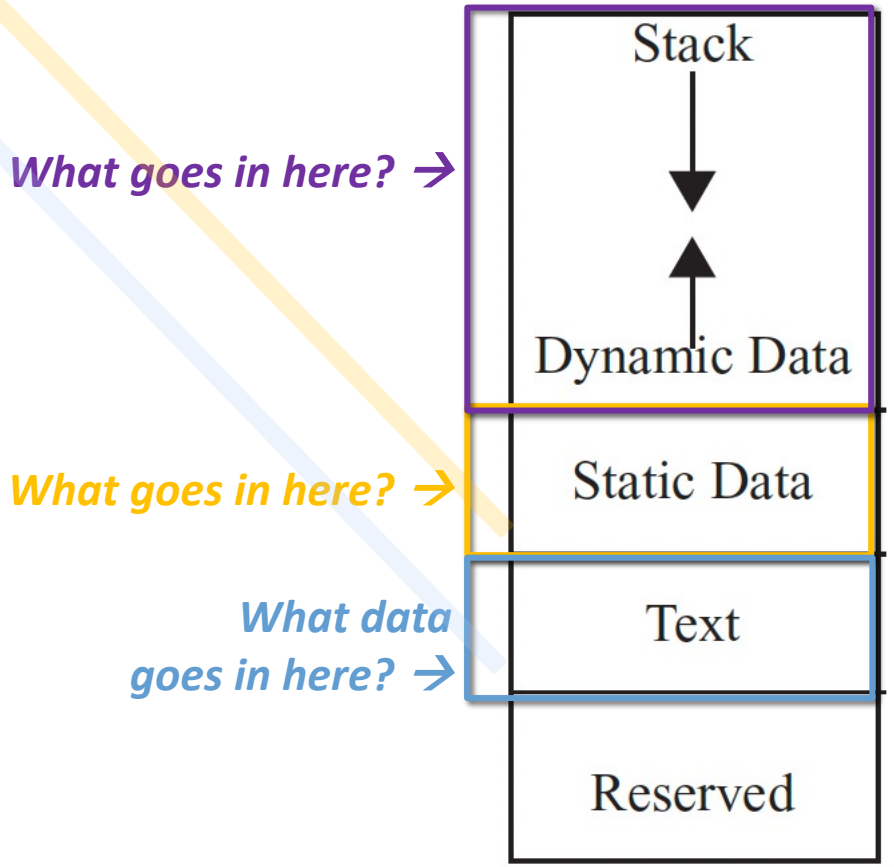
What does this program do?

Worksheet

jimbo.asm

```
.data
name: .asciiz “Jimbo Jones is ”
yold: .asciiz “ years old.”
newline: .asciiz “\n”
.text
main:   li $v0, 4
        la $a0, name    # la = load memory address
        syscall
        li $v0, 1
        li $a0, 15
        syscall
        li $v0, 4
        la $a0, yold
        syscall
        la $a0, newline
        syscall
        li $v0, 10
        syscall
```

What goes in here? → Registers (not part of CPU Memory)



Worksheet

Summary: Using .word *vs* .space

Example1:

```
var: .word 42 5 -6
```

Equivalent example in C++:

```
int var[3] = {42, 5, -6}; // i.e. initialized
```

Example2:

```
var: .space 40
```

Equivalent example in C++:

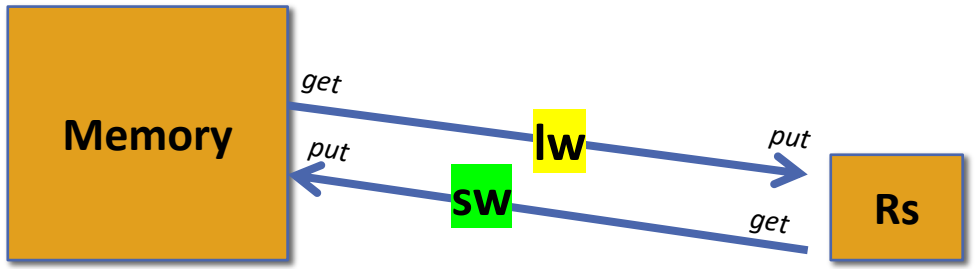
```
int var[10];  // i.e. declared, but not initialized
```

Worksheet

Accessing Memory

2 core instructions for memory access:

- load-word (**lw**) *from memory* ***to registers***
- store-word (**sw**) *from registers* ***to memory***



Go to memory address of ($t5 + 0) and put the value from there into reg. $t0

Get value in reg. $t1 and put that in memory at address ($t6 + 4)

Syntax:

lw $rt, offset($rs)

sw $rt, offset($rs)

Example:

lw $t0, **0**($t5)

sw $t1, **4**($t6)

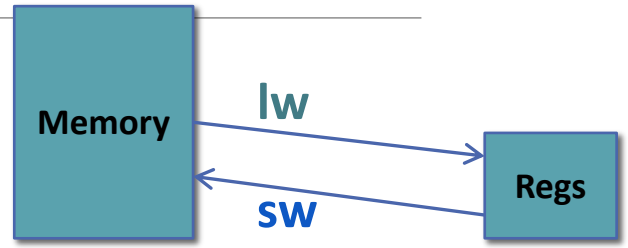
Worksheet

Analysis Example

By the end of the program:

```
a0  =     84
v0  =     10
t0  =     num0
t1  =     num1
t2  =     42
t3  =     ???

*num0   = 42
*num1   = 84
```




```
01: .data
02: num0: .word 42 # num0=42
03: num1: .space 4 # num1=reserved 4-bytes
04:.text
05: main:
06:     la $t0, num0 # t0=num0 (an address value)
07:     la $t1, num1 # t1=num1 (an address value)
08:     lw $t2, 0($t0) # t2=*t0 (the value 42)
09:     lw $t3, 0($t1) # t3=*t1 (the value of ???)
10:
11:     addi $v0, $t2, 42 # v0=42+42=84
12:     sw $v0, 0($t1)   # *t1=v0=84, i.e., *num1=84
```

```
13:     li $v0, 1
14:     lw $a0, 0($t1)
15:     syscall     # prints 84
16:
17:     li $v0, 10
18:     syscall     # quit program
```

To Dos

Turn in current assignment

Look for the new assignment in Canvas!

Do Reading 3 for Wednesday's lecture!!

Study for the Quiz on Wednesday!

Lab on Friday!

</LECTURE>